

A1 · Speaking the Application Layer

CS425 Computer Networks · in class activity · everything runs on onyx.boisestate.edu

TURN THIS IN ON PAPER

before you leave the room
graded pass/fail

Names _____ Date _____

Why you are doing this

Every URL you have ever typed started with a DNS lookup. Today you send one by hand and read the answer that comes back. That is the whole activity: get on Onyx, run `dig`, write down what it tells you. Two rounds. The only way to fail is a blank box.

ROUND 1 · GET ON THE BOX

One machine, everyone on it. If your answer differs from your neighbor's, one of you typed something wrong, because there is no second network to blame.

```
ssh <username>@onyx.boisestate.edu
cat /etc/redhat-release      # which Red Hat
uname -srm                  # kernel and architecture
who | wc -l                  # how many of us are on here right now
```

QUESTION	WHAT IT PRINTED
Red Hat release	
Kernel and architecture	
People logged in right now	

Now find out **which** version of `dig` you have, because that decides which switches work. The other three you will meet in A3; just write the version line down for now.

```
dig -v          curl --version | head -1      nc --version    ping -V
```

TOOL	VERSION LINE
dig	
curl	
nc	
ping	

ROUND 2 · `dig`: READ A DNS ANSWER

DNS is an application layer protocol. A real query goes to a real server and a real answer comes back, and all of it can be inspected. Run the command, then find the four things below in what it printed.

```
dig boisestate.edu
```

FIND THIS IN THE OUTPUT	WRITE IT HERE
The status: field	
The QUESTION section	
ANSWER: type, TTL, address	
Query time:	

Now watch the TTL

Run `dig +noall +answer boisestate.edu` four more times. Then run `dig +noall +answer example.com`, wait three seconds, run it again.

TTL FOR BOISESTATE.EDU	TTL FOR EXAMPLE.COM, TWICE

One of them never moves and the other does, sometimes *upward*. Explain both. You need two facts: a TTL counts down while an answer sits in a cache, and `/etc/resolv.conf` here lists **two** resolvers.

ROUND 2, CONTINUED · THE SWITCHES THAT MATTER

One sentence each: what did this switch change about the output?

COMMAND	WHAT IT CHANGED
<code>dig +short boisestate.edu</code>	
<code>dig +noall +answer boisestate.edu</code>	
<code>dig boisestate.edu MX</code>	
<code>dig boisestate.edu NS</code>	
<code>dig boisestate.edu TXT</code>	
<code>dig boisestate.edu AAAA</code>	
<code>dig @1.1.1.1 boisestate.edu ← fails</code>	
<code>dig +trace boisestate.edu ← fails</code>	

Also try `dig +tcp boisestate.edu` (same query over TCP) and `dig +stats boisestate.edu` (timing, server used, message size). Note anything that surprised you: _____

The two that fail, and why that is the interesting part

`@1.1.1.1` times out; `+trace` stops after one step. Neither is broken software: both need to send DNS queries **directly to a server out on the internet**, and this network does not allow it. What do the two failures have in common, and what is sitting between Onyx and the internet that causes both?

The trap

```
dig nosuchthing.boisestate.edu
dig +short nosuchthing.boisestate.edu
```

What was the **status:** field? And what exactly did the second command print?

In one sentence: why is `+short` dangerous when you are debugging, even though it is convenient when you are scripting?

If you finish early: `dig -x <address>` does the query backwards, address in, name out. Run it on the address `dig +short boisestate.edu` printed. What came back, and is it the name you started with? _____

Graded pass/fail on a serious attempt at both rounds. Hand this in before you leave, with every name at the top.